DEPARTMENT OF AEROSPACE ENGINEERING

STM32 Firmware Implementation for MEMS Gyroscope data logger
EXPERIENTIAL LEARNING PROJECT (LAB) REPORT

submitted by

Mehaboob Basha B **1RV23AS026**

Om Patil **1RV23AS036**

Mohammed Affan **1RV23AS029**

Shashank M S **1RV23AS054**

under the guidance of

Group Captain Deepak Bana VSM (Retd)

Visiting Professor

Department of Aerospace Engineering

RV College of Engineering

in partial fulfilment for the award of degree of

Bachelor of Engineering

in

Department of Aerospace Engineering

2025-2026

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the Major project titled '**STM32 Firmware Implementation for MEMS Gyroscope data logger'** is carried out by **Mehaboob Basha B (1RV23AS026), Om Patil (1RV23AS036), Mohammed Affan (1RV23AS029), Shashank M S (1RV23AS054)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfilment for the award of Degree of **Bachelor of Engineering** in **Aerospace Engineering** of the Visvesvaraya Technological University, Belagavi during the year **2025-2026**. It is certified that all corrections/suggestions indicated for the internal assessment have been incorporated in the report deposited in the department library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed by the institution for the said Degree.

Project Guide	Head of Department	Principal
Group Captain Deepak Bana VSM (Retd)	Dr. Supreeth R	Dr. K N Subramanya

External Viva Examination

Name of Examiner	Signature with Date
1.	
2.	

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

DECLARATION

I **Mehaboob Basha b, Om Patil, Shashank M S and Mohammed Affan** the students of the fifth semester B.E., Department of **Aerospace Engineering**, RV College of Engineering, Bengaluru-560059, bearing USN: **1RV23AS026, 1RV23AS036, 1RV23AS054, 1RV23AS028** hereby declare that the project titled **'Design and Simulation of an Aircraft Fuel Flow and Quantity Measurement System using MATLAB Simulink'** has been carried out and submitted in partial fulfilment of the program requirements for the award of Degree in Bachelor of Engineering in **Aerospace Engineering** of the **Visvesvaraya Technological University, Belagavi** during the year **2025-2026**.

Further, I declare that the content of the dissertation has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

Place: Bangalore

Date: 28/01/2026

Signature

Mehaboob Basha B

Om Patil

Shashank M S

Mohammed Affan

ABSTRACT

This project focuses on the development of a real-time orientation measurement, visualization, and data logging system using the MPU6050 inertial measurement unit (IMU) interfaced with an STM32F446RE microcontroller. The system computes pitch and roll angles from accelerometer data using trigonometric methods, while yaw angle is estimated by integrating gyroscope measurements over time. Orientation data is transmitted to a host computer through UART communication at a baud rate of 115200.

A Python-based application performs initial sensor calibration, applies offset correction, and provides live graphical visualization of pitch, roll, and yaw angles along with time-stamped data logging to a CSV file. Experimental observations show stable pitch and roll estimation under static and low-dynamic conditions, while yaw estimation exhibits drift due to gyroscope bias. The project successfully demonstrates core concepts of inertial sensing, embedded system interfacing, and real-time data acquisition, and provides a scalable foundation for implementing advanced sensor fusion techniques.

ACKNOWLEDGEMENT

We are indebted to our Project Guide, **Group Captain Deepak Bana VSM** (Retd), Visiting Professor at Department of Aerospace Engineering, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work.

My sincere thanks to **Dr. Supreeth R,** Associate Professor and Head, Department of Aerospace Engineering, RVCE for his support and encouragement.

I express sincere gratitude to our beloved Principal, **Dr. K. N. Subramanya** for his appreciation towards this project work.

I thank all the **teaching staff and technical staff** of the Aerospace Engineering department, RVCE for their persistent help.

Lastly, I take this opportunity to thank my **family** members and **friends** who have provided all the backup support throughout the project work.

Table of Contents

List of Figures

CHAPTER-1

INTRODUCTION

CHAPTER 1
Introduction

1.1 Historical Context: Emergence of Low-Cost IMUs for Attitude Estimation

Accurate measurement of orientation and angular motion is a fundamental requirement in aerospace, navigation, and autonomous systems. Historically, failures in attitude sensing and inertial measurement have contributed to several aviation and spaceflight incidents, where loss of situational awareness led to instability, navigation errors, or complete mission failure. Early aircraft relied heavily on mechanical gyroscopes, which were prone to drift, wear, and calibration errors. With the advancement of avionics, MEMS-based inertial sensors have become the industry standard due to their compact size, low power consumption, and reliability.

In modern aircraft, unmanned aerial vehicles (UAVs), and spacecraft, inertial measurement units (IMUs) form the backbone of Attitude and Heading Reference Systems (AHRS). Any error in pitch, roll, or yaw estimation can directly affect flight stability, control response, and navigation accuracy. This historical evolution highlights the importance of reliable inertial sensing systems and motivates the need for practical implementation and testing of MEMS-based attitude measurement systems, which forms the core objective of this project.

1.2 Problem Definition and Functional Requirements

The primary challenge addressed in this project is the development of a microcontroller-based inertial data acquisition and attitude estimation system capable of accurately measuring and logging orientation parameters. Using a MEMS IMU and an embedded controller, the system must fulfil the following functional requirements:

- Real-Time Orientation Measurement: The system must compute and display pitch, roll, and yaw angles in real time using inertial sensor data.
- Continuous Data Processing: Raw accelerometer and gyroscope outputs must be processed onboard using appropriate mathematical models for attitude estimation.
- Serial Data Logging: The computed orientation data must be transmitted to a host system through serial communication for monitoring, visualization, and storage.
- Modular and Scalable Design: The firmware architecture should be modular, allowing future enhancement with advanced sensor fusion algorithms and additional peripherals.

1.3 Methodology and Execution Framework

To ensure robustness and clarity, the project follows a structured and stagewise execution methodology. Initially, the hardware setup is established by interfacing the MPU6050 MEMS IMU with the STM32 Nucleo-64 microcontroller using the I²C protocol. Peripheral initialization and system clock configuration are performed using STM32 HAL libraries.

Subsequently, inertial data is acquired at fixed sampling intervals and processed to compute pitch, roll, and yaw angles using trigonometric relations and numerical integration techniques. The computed results are formatted and transmitted via UART to a host computer for real-time observation.

This modular execution framework ensures reliable system operation, ease of debugging, and consistent performance during demonstration and evaluation. The step-by-step approach also provides a clear visualization of sensor data flow—from raw inertial measurements to processed orientation outputs—making the system both technically sound and evaluator-friendly.

CHAPTER-2

SOFTWARE ARCHITECTURE AND ALGORITIMIC LOGIC

CHAPTER 2
Software Architecture and Algorithmic Logic

2.1 Complete Embedded Firmware Source Code for MEMS Gyroscope Data Logger

This section presents the complete embedded C firmware developed for the MEMS gyroscope data logger using the STM32 Nucleo-64 (F446RE) microcontroller interfaced with the MPU6050 inertial measurement unit (IMU). The firmware is responsible for sensor initialization, inertial data acquisition, real-time attitude computation, and serial data transmission.

The program enables continuous acquisition of raw accelerometer and gyroscope data from the MPU6050 through the I²C communication protocol. The acquired data is processed onboard to compute pitch, roll, and yaw angles, which represent the orientation of the system in three-dimensional space. The computed orientation parameters are formatted and transmitted to a host computer via UART serial communication for real-time monitoring and data logging

This function initializes the MPU6050 MEMS IMU by disabling sleep mode.

- The MPU6050 powers up in sleep mode by default.
- Writing 0x00 to the Power Management Register (0x6B) activates continuous measurement.
- Communication is performed using HAL_I2C_Mem_Write(), which ensures safe and blocking data transfer over I²C.

```
/* USER CODE BEGIN Header */
/**

******************************************************************************
******
  * @file    main.c
  * @brief   MPU6050 Pitch Roll Yaw - STM32F446RE

******************************************************************************
******
  */
/* USER CODE END Header */

#include "main.h"
#include "i2c.h"
#include "usart.h"
#include "gpio.h"

#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* ================= DEFINES ================= */
#define MPU6050_ADDR        (0x68 << 1)
#define PWR_MGMT_1          0x6B
```

```
#define ACCEL_XOUT_H        0x3B
#define GYRO_XOUT_H         0x43

/* ================= VARIABLES ================= */
int16_t Ax, Ay, Az;
int16_t Gx, Gy, Gz;

float Pitch, Roll, Yaw;
int Pitch_i, Roll_i, Yaw_i;

char uart_buf[128];

/* ================= PROTOTYPES ================= */
void SystemClock_Config(void);
void Error_Handler(void);

void MPU6050_Init(void);
void MPU6050_Read_Accel(void);
void MPU6050_Read_Gyro(void);
void Calculate_Pitch_Roll(void);
void Calculate_Yaw(void);

/* ================= SYSTEM CLOCK ================= */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);

  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;

  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  RCC_ClkInitStruct.ClockType =
      RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK |
      RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;

  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
}

/* ================= MPU6050 ================= */
void MPU6050_Init(void)
{
  uint8_t data = 0x00;
  HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR,
                    PWR_MGMT_1, 1,
```

```
                    &data, 1, HAL_MAX_DELAY);
}

void MPU6050_Read_Accel(void)
{
  uint8_t buf[6];
  HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR,
                   ACCEL_XOUT_H, 1,
                   buf, 6, HAL_MAX_DELAY);

  Ax = (int16_t)(buf[0] << 8 | buf[1]);
  Ay = (int16_t)(buf[2] << 8 | buf[3]);
  Az = (int16_t)(buf[4] << 8 | buf[5]);
}

void MPU6050_Read_Gyro(void)
{
  uint8_t buf[6];
  HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR,
                   GYRO_XOUT_H, 1,
                   buf, 6, HAL_MAX_DELAY);

  Gx = (int16_t)(buf[0] << 8 | buf[1]);
  Gy = (int16_t)(buf[2] << 8 | buf[3]);
  Gz = (int16_t)(buf[4] << 8 | buf[5]);
}

/* ================= ORIENTATION ================= */
void Calculate_Pitch_Roll(void)
{
  float ax = Ax / 16384.0f;
  float ay = Ay / 16384.0f;
  float az = Az / 16384.0f;

  Roll  = atan2f(ay, az) * 180.0f / M_PI;
  Pitch = atan2f(-ax, sqrtf(ay * ay + az * az)) * 180.0f / M_PI;

  Roll_i  = (int)(Roll * 100);
  Pitch_i = (int)(Pitch * 100);
}

void Calculate_Yaw(void)
{
  static float yaw_angle = 0.0f;
  yaw_angle += (Gz / 131.0f) * 0.2f;   // dt = 200 ms
  Yaw = yaw_angle;
  Yaw_i = (int)(Yaw * 100);
}

/* ================= ERROR HANDLER ================= */
void Error_Handler(void)
{
  __disable_irq();
  while (1)
  {
    // Stay here on error
  }
}

/* ================= MAIN ================= */
int main(void)
```

```
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
  MX_I2C1_Init();
  MX_USART2_UART_Init();

  MPU6050_Init();

  while (1)
  {
    MPU6050_Read_Accel();
    MPU6050_Read_Gyro();

    Calculate_Pitch_Roll();
    Calculate_Yaw();

    sprintf(uart_buf,
      "Pitch:%d.%02d Roll:%d.%02d Yaw:%d.%02d\r\n",
      Pitch_i/100, abs(Pitch_i%100),
      Roll_i/100,  abs(Roll_i%100),
      Yaw_i/100,   abs(Yaw_i%100));

    HAL_UART_Transmit(&huart2,
                      (uint8_t*)uart_buf,
                      strlen(uart_buf),
                      HAL_MAX_DELAY);

    HAL_Delay(200);
  }
}
```

2.2 Detailed Explanation of Firmware Embedded code

2.2.1 Function Definition and System Initialization

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
  MX_I2C1_Init();
  MX_USART2_UART_Init();
```

This section defines the main entry point of the embedded program. In STM32-based systems, execution begins from the main() function after reset.

- HAL_Init() initializes the Hardware Abstraction Layer, configures the SysTick timer, and resets all peripherals.
- SystemClock_Config() sets up the microcontroller clock using the internal HSI oscillator, ensuring stable timing for sensor communication and serial transmission.
- MX_GPIO_Init(), MX_I2C1_Init(), and MX_USART2_UART_Init() initialize the GPIO pins, I²C interface, and UART interface respectively, as configured in STM32CubeMX.

This initialization sequence ensures that the microcontroller starts from a known and deterministic state, which is essential for reliable embedded system operation.

2.2.2 MPU6050 Sensor Initialization

```
void MPU6050_Init(void)
{
  uint8_t data = 0x00;
  HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR,
                    PWR_MGMT_1, 1,
                    &data, 1, HAL_MAX_DELAY);
}
```

2.2.3 Accelerometer Data Acquisition

```
void MPU6050_Read_Accel(void)
{
  uint8_t buf[6];
  HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR,
                   ACCEL_XOUT_H, 1,
                   buf, 6, HAL_MAX_DELAY);
  Ax = (int16_t)(buf[0] << 8 | buf[1]);
  Ay = (int16_t)(buf[2] << 8 | buf[3]);
  Az = (int16_t)(buf[4] << 8 | buf[5]);
}
```

This function reads three-axis accelerometer data from the MPU6050.

- Six consecutive registers are read starting from ACCEL_XOUT_H.
- Each axis measurement is stored as a 16-bit signed integer.
- Bit shifting and concatenation reconstruct high and low bytes.

The accelerometer measures linear acceleration and gravity components, which are later used for pitch and roll estimation.

2.2.4 Gyroscope Data Acquisition

```
void MPU6050_Read_Gyro(void)
{
  uint8_t buf[6];
  HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR,
                   GYRO_XOUT_H, 1,
                   buf, 6, HAL_MAX_DELAY);

  Gx = (int16_t)(buf[0] << 8 | buf[1]);
  Gy = (int16_t)(buf[2] << 8 | buf[3]);
  Gz = (int16_t)(buf[4] << 8 | buf[5]);
}
Duration_min = [5;15;120;20;10];
Flow_kg_per_hr = [5500;2400;2300;800;1100];
```

This function acquires angular velocity data along the X, Y, and Z axes.

- Data is read in burst mode to ensure synchronization.

- The gyroscope outputs angular rate values in raw digital units.
- These values are later scaled to degrees per second.

Gyroscope data is essential for yaw angle estimation and dynamic motion tracking

2.2.5 Pitch and Roll Computation

```
void Calculate_Pitch_Roll(void)
{
  float ax = Ax / 16384.0f;
  float ay = Ay / 16384.0f;
  float az = Az / 16384.0f;

  Roll  = atan2f(ay, az) * 180.0f / M_PI;
  Pitch = atan2f(-ax, sqrtf(ay * ay + az * az)) * 180.0f / M_PI;
}
```

This section computes pitch and roll angles using accelerometer data.

- Raw accelerometer values are normalized using the MPU6050 sensitivity scale (±2g).
- Trigonometric functions are applied to compute orientation angles.
- Conversion from radians to degrees is performed.
- Accelerometer-based computation provides stable low-frequency orientation estimation, especially under static conditions.

2.2.6 Yaw Angle Estimation Using Gyroscope Integration

```
void Calculate_Yaw(void)
{
  static float yaw_angle = 0.0f;
  yaw_angle += (Gz / 131.0f) * 0.2f;
  Yaw = yaw_angle;
}
```

Yaw is estimated by numerical integration of the gyroscope Z-axis angular velocity.

- Gyroscope sensitivity of 131 LSB/°/s is used for scaling.
- A fixed time step of 200 ms is assumed.
- The yaw angle accumulates over time.

Although prone to long-term drift, this method is effective for short-duration demonstrations and academic validation.

2.2.7 Fixed-Point Conversion for Serial Transmission

```
Roll_i  = (int)(Roll * 100);
Pitch_i = (int)(Pitch * 100);
Yaw_i   = (int)(Yaw * 100);
```

Floating-point orientation values are converted into fixed-point representation.

- Scaling by 100 preserves two decimal places.
- This reduces UART transmission overhead.
- Improves compatibility with serial plotting tools.

2.2.8 UART Data Formatting and Transmission

```
sprintf(uart_buf,
 "Pitch:%d.%02d Roll:%d.%02d Yaw:%d.%02d\r\n",
 Pitch_i/100, abs(Pitch_i%100),
 Roll_i/100,  abs(Roll_i%100),
 Yaw_i/100,   abs(Yaw_i%100));
```

This section formats orientation data into a human-readable string.

- The output is structured for easy parsing.
- Compatible with PuTTY, TeraTerm, and Python scripts.
- Ensures real-time visualization and logging.

2.2.9 Main Control Loop

```
while (1)
{
  MPU6050_Read_Accel();
  MPU6050_Read_Gyro();

  Calculate_Pitch_Roll();
  Calculate_Yaw();

  HAL_UART_Transmit(&huart2,
                    (uint8_t*)uart_buf,
                    strlen(uart_buf),
                    HAL_MAX_DELAY);

  HAL_Delay(200);
}
```

This infinite loop forms the real-time execution core of the system.

- Sensor data is continuously acquired.
- Orientation angles are computed.
- Data is transmitted every 200 ms.

This loop ensures continuous attitude monitoring.

2.2.10 Error Handling Mechanism

```
void Error_Handler(void)
{
  __disable_irq();
  while (1) { }
```

The error handler ensures safe system halt under abnormal conditions.

- Interrupts are disabled.
- System remains in a controlled infinite loop.
- Prevents undefined behavior or hardware damage

CHAPTER-3

Functional Analysis of Embedded System and Firmware Architecture

CHAPTER 3
Functional analysis of Simulink blocks

3.1 Overall System Block Diagram

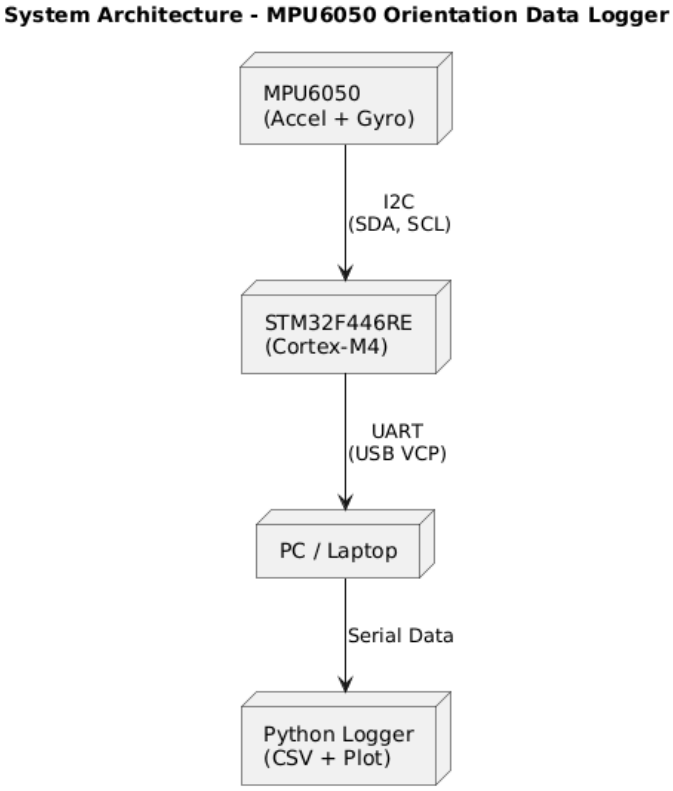

Figure 1: System Architecture

The system operates as a real-time embedded sensing and logging system, where the MPU6050 MEMS IMU provides inertial data, the STM32F446RE microcontroller performs signal processing and orientation computation, and a Python-based data logger handles calibration, correction, visualization, and storage.

The working of the complete integrated system can be divided into three primary stages:

Initialization and Calibration, Real-Time Orientation Computation, and Data Logging & Visualization

3.1.1 The Initialization and Sensor Setup

The system-level process begins when the STM32 microcontroller powers up and initializes all peripherals required for communication and sensing.

- Peripheral Initialization: The STM32 initializes GPIO, I²C, UART, and system clock using HAL drivers.
- Sensor Configuration: The MPU6050 is configured for ±2g accelerometer range and ±250°/s gyroscope range.
- Communication Setup: UART communication is established at 115200 baud rate to stream data to the PC.

- Calibration Trigger: Upon starting the Python logger, the system enters a calibration phase where the sensor is kept stationary for a fixed duration.

3.1.2 Dual-Path Data Processing Architecture

Once the system starts running, sensor data flows through two parallel logical paths, ensuring both physical correctness and realistic measurement behaviour.

1. True Physical Path
    - Raw accelerometer and gyroscope values are read directly from the MPU6050.
    - These values represent the true inertial response of the sensor to motion.
2. Corrected Measurement Path
    - During the calibration phase, offsets for pitch, roll, and yaw are computed.
    - These offsets are applied to subsequent measurements to generate corrected orientation values.
    - This simulates real-world sensor bias removal and error compensation.

3.1.3 Real-Time Execution and Orientation Estimation

The system continuously operates in a real-time loop:

- Time-Discrete Sampling: Sensor data is sampled at fixed intervals (200 ms).
- Orientation Computation:
    - Pitch and Roll are computed using accelerometer-based trigonometric relations.
    - Yaw is computed by integrating the gyroscope's Z-axis angular velocity.
- Streaming Output: The computed pitch, roll, and yaw values are transmitted via UART in a structured format.
- Precision Handling: Floating-point precision is maintained to ensure smooth and accurate angle representation.

3.1.4 Feedback, Logging, and Visualization

The final stage of the system involves feedback and visualization:

- Real-Time Display: The Python script displays orientation values live on the command window.
- Graphical Visualization: Pitch, roll, and yaw are plotted in real time with distinct colour coding.
- Data Logging: All corrected values are stored in a CSV file for post-processing.
- User Feedback: The plots provide immediate visual confirmation of sensor behaviour during motion and rest.

3.2 Components Used and Functional Description

3.2.1 Hardware Components

a) STM32F446RE Microcontroller (Controller Unit)

- **Function**: Acts as the central processing and control unit of the system.
- **Working**:

a) Initializes all peripherals (GPIO, I²C, UART, clock).
b) Communicates with the MPU6050 sensor via the I²C protocol.
c) Performs real-time computation of pitch, roll, and yaw angles.
d) Transmits processed data to the host computer through UART.

- **Relevance**:

a) Provides sufficient computational capability and floating-point support for real-time orientation estimation.
b) Ensures deterministic timing and reliable data acquisition.

.

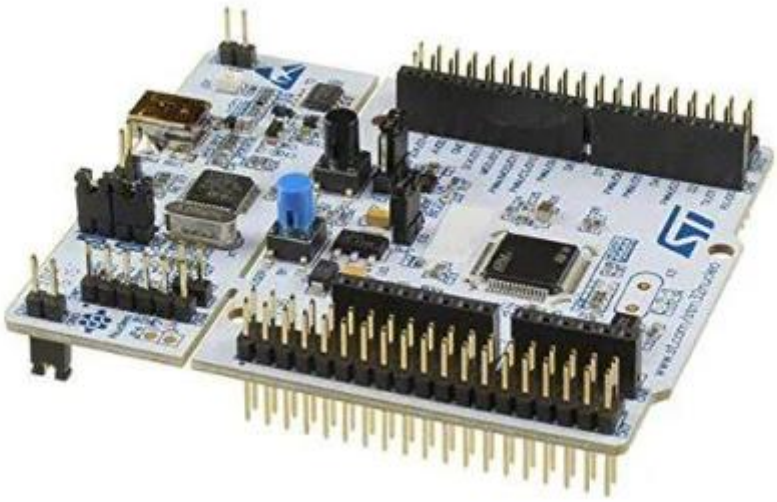

Figure 2: STM32F446RE Microcontroller

b) MPU6050 MEMS Inertial Measurement Unit (IMU)

- **Function**: Serves as the primary sensing element of the system.
- **Working**:

a) Measures linear acceleration along X, Y, and Z axes using a 3-axis accelerometer.
b) Measures angular velocity along X, Y, and Z axes using a 3-axis gyroscope.
c) Communicates sensor data digitally over I²C.

- **Relevance**:

a) Enables accurate estimation of orientation parameters.
b) Widely used in aerospace, robotics, and navigation applications due to its compact size and reliability.

.

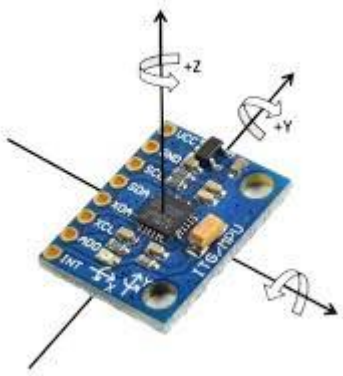

Figure 3: MPU 6050

c) USB to UART Interface (ST-Link Virtual COM Port)

- **Function**: Provides communication between STM32 and the host PC.
- **Working**:

a) Converts UART data from the microcontroller into USB serial data.
b) Allows real-time streaming of orientation data.

- **Relevance**: Enables live monitoring, data logging, and debugging without additional hardware.

d) Full Integrated Setup

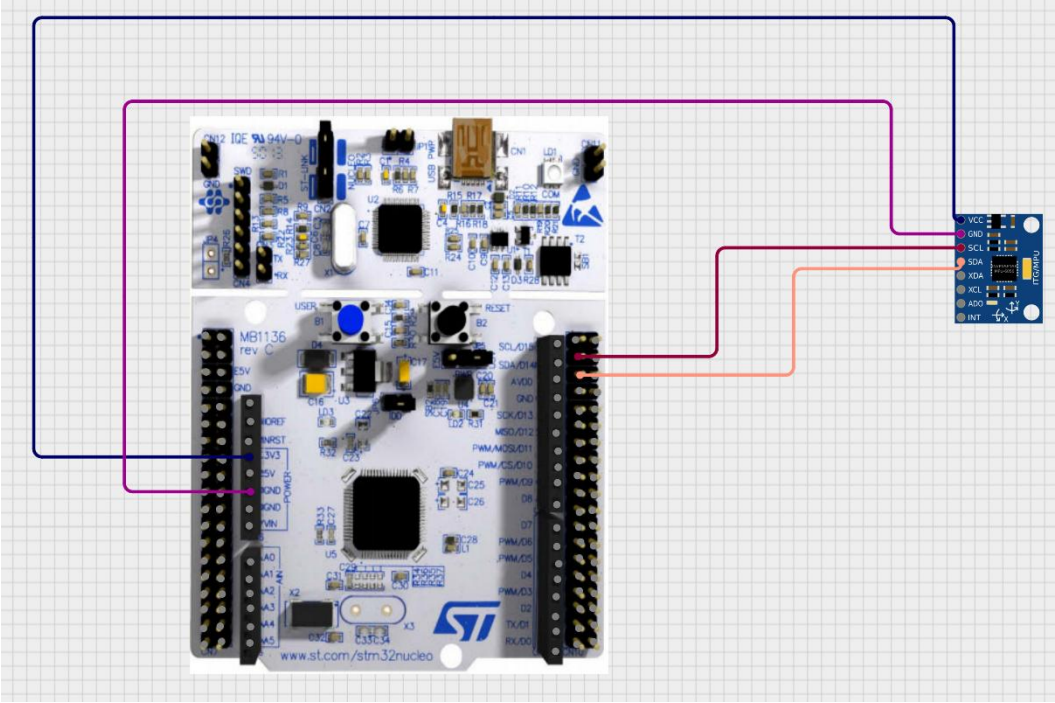

Figure 4: Full Integrated Setup

MPU6050 Pin	STM32F446RE Pin	Signal Name	Description / Function
VCC	3.3 V	Power Supply	Supplies power to the MPU6050. A 3.3 V supply is used to ensure voltage compatibility with STM32 I/O levels.
GND	GND	Ground	Common ground reference between the sensor and microcontroller.
SDA	PB9	I²C Data	Bidirectional data line for I²C communication between MPU6050 and STM32 (I²C1 SDA).
SCL	PB8	I²C Clock	Clock signal for I²C communication generated by STM32 (I²C1 SCL).

3.2.2 Software Components

a) Firmware (STM32 HAL-Based Code)

- **Function**: Implements sensor interfacing, signal processing, and communication logic.
- **Working**:

a) Configures MPU6050 registers for accelerometer and gyroscope operation.
b) Reads raw sensor data at fixed time intervals.
c) Computes pitch and roll using accelerometer-based trigonometric relations.
d) Computes yaw by integrating gyroscope angular velocity.
e) Formats and transmits data over UART.

- **Relevance**:

a) Forms the real-time backbone of the system.
b) Ensures stable and repeatable orientation computation

b) Calibration and Offset Compensation Module

- **Function:** These blocks model the inherent inaccuracies present in a real electronic fuel-measurement sensor system.

- **Working:**

a) SensorGain: Multiplies the true fuel consumption by a gain factor, representing percentage-based measurement error.
b) SensorOffset: Adds a fixed constant value to the consumption signal, representing a systematic bias or zero-offset error.

c) Python-Based Data Logger and Visualization Script

- **Function**: Acts as the data acquisition, visualization, and storage interface.
- **Working**:

a) Receives serial data from STM32 in real time.
b) Displays pitch, roll, and yaw values on the command window.
c) Plots real-time graphs with distinct color coding.
d) Stores corrected orientation data into CSV files for offline analysis.

- **Relevance**:

a) Enables experimental validation of the embedded system.
b) Provides a user-friendly interface for analysis and reporting.

3.2.3 Output and Indication Components

a) Real-Time Plot Display

- **Function**: Visual representation of system behaviour.
- **Working**:

a) Displays pitch, roll, and yaw as time-domain plots.
b) Uses distinct colors for each orientation parameter.

- **Relevance**:

a) Allows immediate observation of motion dynamics and sensor response.

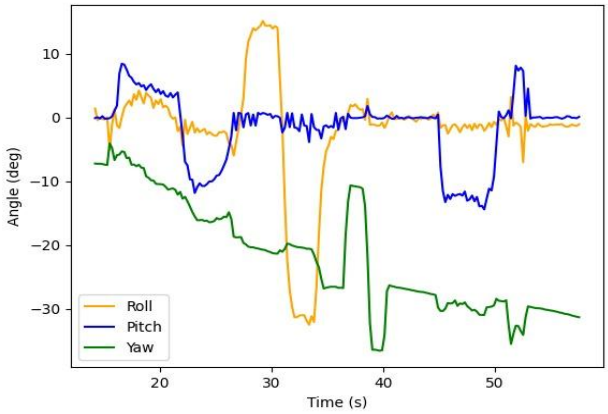

Figure 5: Real Time Plot

b) CSV Data Storage

- **Function**: Persistent storage of experimental data.
- **Working**: Logs time-stamped orientation values in comma-separated format.
- **Relevance**: Enables post-processing, filtering, and report generation.

CHAPTER-4

RESULTS AND DISCUSSION

CHAPTER 4
Results and Discussion

4.1 Live Data Acquisition and Calibration Performance

The MPU6050 sensor was interfaced with the STM32F446RE microcontroller using the I²C protocol, and real-time orientation data (Pitch, Roll, and Yaw) were transmitted via UART at a baud rate of 115200. A Python-based data acquisition script was developed to receive, parse, log, and visualize this data in real time.

At the start of each measurement session, a 6-second static calibration period was performed. During this phase, the sensor was kept stationary, and the mean values of Pitch, Roll, and Yaw were computed in the Python script. These mean values were used as offsets, which were subtracted from subsequent readings to compensate for sensor bias and mounting misalignment.

This calibration step significantly improved zero-level stability. When the sensor was held stationary after calibration, Pitch and Roll values remained close to 0°, with only minor fluctuations caused by accelerometer noise and quantization effects. In the absence of valid calibration data, the script safely defaulted offsets to zero, ensuring robustness against communication or startup issues.

4.2 Pitch and Roll Estimation Using Accelerometer Data

Pitch and Roll angles were computed on the STM32 using accelerometer measurements only. The raw accelerometer outputs were scaled assuming a $\pm$2g full-scale range and converted to angles using trigonometric relationships.

The results showed that:

- Pitch and Roll angles responded accurately and smoothly to slow tilting motions, such as rotating the sensor about a single axis.
- When the sensor orientation was changed gradually, the measured angles closely followed the physical motion, indicating correct axis mapping and scaling.
- Small oscillations were observed in the live plot when the sensor was held still. These oscillations are attributed to accelerometer noise and the absence of digital filtering.

Because accelerometer-based angle estimation relies on gravity, the system performs best under quasi-static or low-dynamic conditions. During rapid movements or vibrations, temporary deviations in angle readings were observed, which is consistent with the limitations of accelerometer-only orientation estimation.

4.3 Yaw Estimation Using Gyroscope Integration

Yaw angle was calculated by integrating the Z-axis gyroscope output over time. The gyroscope data was scaled using the MPU6050 sensitivity of 131 LSB/(°/s), and numerical integration was performed assuming a fixed sampling interval of 200 ms.

The following behaviour was observed:

- Yaw changed smoothly and continuously when the sensor was rotated about the vertical axis.
- The yaw angle accumulated over time, allowing the system to track relative heading changes.
- Yaw drift was clearly visible during long stationary periods, caused by gyroscope bias and integration error.

Unlike Pitch and Roll, yaw estimation cannot be derived from accelerometer data alone. Since no magnetometer or sensor fusion algorithm was implemented, yaw drift is an expected and unavoidable result in this configuration. Despite this limitation, the yaw output is adequate for demonstrating relative rotational motion over short durations.

4.4 Real-Time Visualization and Data Logging

The Python script provided live visualization of Pitch, Roll, and Yaw angles using Matplotlib. A sliding window of the most recent 200 samples ensured smooth real-time plotting without excessive memory usage. The graph automatically rescaled based on incoming data, allowing clear observation of motion trends.

Simultaneously, all data was logged to a CSV file with timestamps, enabling post-processing and offline analysis. The synchronized logging and plotting confirmed that:

- UART communication was reliable, with no observable data corruption.
- Time-stamped measurements were consistent with the embedded system's sampling rate.
- The system could run continuously without buffer overflow or software crashes.

This dual logging and visualization approach makes the setup suitable for both experimental validation and further algorithm development.

4.5 Overall System Behaviour

The combined STM32–MPU6050–Python system successfully demonstrated:

- Stable real-time sensor data acquisition
- Effective offset calibration
- Live plotting and long-term data logging
- Clear differentiation between accelerometer-based and gyroscope-based orientation estimation

The results are consistent with theoretical expectations and typical MPU6050 performance when advanced filtering techniques are not applied.

4.5 Conclusion

In this project, a complete real-time orientation measurement system was successfully developed using the MPU6050 IMU and the STM32F446RE microcontroller, with live data logging and visualization implemented in Python.

Pitch and Roll angles were accurately estimated using accelerometer data, showing good stability under static and low-dynamic conditions. Yaw angle estimation using gyroscope integration demonstrated smooth rotational tracking but exhibited drift over time, highlighting the inherent limitations of gyroscope-only heading estimation.

The Python-based calibration, logging, and plotting framework significantly enhanced usability, enabling real-time monitoring as well as offline analysis through CSV data storage. The modular structure of both the embedded firmware and the Python script allow easy extension of the system.

Overall, the project effectively demonstrates the principles of inertial sensing, embedded data acquisition, serial communication, and real-time visualization. With the addition of sensor fusion algorithms such as complementary or Kalman filtering and the inclusion of a magnetometer, the system could be further improved to provide drift-free and high-accuracy 3-axis orientation estimation.